

Introduction to Deep Learning

Magda Gregorová

magda.gregorova@thws.de

March 2026

Licensed according to [CC-BY-SA 4.0](#)

Contents

1	Generalization & Regularization	2
1.1	Generalisation	2
1.1.1	True Goal and Generalisation Gap	2
1.1.2	Train, Validation, and Test Split	2
1.1.3	Overfitting and Underfitting	3
1.2	Generalisation Paradox	3
1.2.1	Bias-Variance Tradeoff	4
1.2.2	Limits of the Classical View	4
1.2.3	Many Good Minima	4
1.2.4	Flat vs Sharp Minima	4
1.2.5	Double Descent	5
1.3	Regularisation Techniques	6
1.3.1	Weight Decay	6
1.3.2	Dropout	6
1.3.3	<code>model.train()</code> and <code>model.eval()</code>	8
1.3.4	Data Augmentation	8
1.3.5	Early Stopping	9
1.4	Training Stability	10
1.4.1	Internal Covariate Shift and Batch Normalisation	10
1.4.2	Vanishing and Exploding Gradients	12
1.4.3	Dying ReLU and Activation Functions	13
1.5	Embracing Randomness	13
	Acknowledgements	14
	References	15

1 Generalization & Regularization

These notes accompany the seventh lecture. Sessions 1–6 built the full machinery for training deep networks: forward pass, backpropagation, optimisation, and the PyTorch API. Throughout, we minimised a training loss and measured progress by how low it went. This session steps back and asks a more fundamental question: is minimising training loss the right objective? The answer is no - the true goal is performance on new, unseen data from the population. We develop the concept of generalisation, examine why deep networks generalise surprisingly well despite being massively overparameterised, and then study the main techniques for ensuring generalisation in practice.

1.1 Generalisation

We begin by formalising what a good model actually means and establishing the tools for measuring it.

► Session 7: Where We Are, True Goal, Train / Validation / Test Split, Overfitting and Underfitting, What To Do

1.1.1 True Goal and Generalisation Gap

Throughout this course we have trained models by minimising the empirical loss on a training dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta; \mathcal{D}) = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}^{(i)}), y^{(i)}). \quad (1.1)$$

But what we actually want is a model that performs well on **new, unseen examples drawn from the same population**. The true objective is the **population loss**:

$$\mathcal{L}_{\text{pop}}(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}} [\ell(f_{\theta}(\mathbf{x}), y)], \quad (1.2)$$

where p_{data} is the true data-generating distribution. This is an **expectation** - the average loss over all possible examples from the population. On some examples the model will do better than average, on others worse; this reflects natural variation in the data and is unavoidable. Optimising the average is the right objective. The training set $\mathcal{D}$ is a finite sample from p_{data} , and minimising $\mathcal{L}(\theta; \mathcal{D})$ is only a proxy for minimising $\mathcal{L}_{\text{pop}}(\theta)$.

The **generalisation gap** quantifies how well the proxy serves the true objective:

$$\text{generalisation gap} = \mathcal{L}_{\text{pop}}(\theta) - \mathcal{L}_{\text{train}}(\theta) \geq 0. \quad (1.3)$$

A model with a small generalisation gap has learned something genuinely useful about the data-generating process. A model with a large gap has overfit - it has memorised properties of the training set that do not hold for the population.

1.1.2 Train, Validation, and Test Split

In practice, a dataset is divided into three non-overlapping subsets with distinct roles:

Training set - used to fit model parameters by minimising $\mathcal{L}(\theta; \mathcal{D}_{\text{train}})$. The model sees this data repeatedly during training.

Validation set - used to evaluate the model during development: selecting architecture, tuning hyperparameters, and deciding when to stop training. The model does

not train on validation data, but validation performance influences decisions about the model. It is therefore touched many times during development.

Test set - used for final, honest evaluation of the deployed model. It is a held-out sample from p_{data} used to estimate $\mathcal{L}_{\text{pop}}$ - a **measurement tool, not the true objective**. Repeatedly evaluating on the test set and adjusting the model is a form of **overfitting the test set**: the test loss becomes an optimistic estimate of true population performance. This is why the test set must be touched only once, at the very end of development, and must not influence any modelling decision.

Remark 1. The key requirement for validation and test sets is that they are **representative of the population** - they should reflect the full variation in p_{data} . A more complex population requires larger sets to estimate performance reliably. At the same time, every example allocated to validation or test is unavailable for training, so there is a tradeoff. Typical splits of 80/10/10 or 70/15/15 are reasonable starting points, but the right split depends on dataset size and population complexity.

1.1.3 Overfitting and Underfitting

Two failure modes arise from the training-population gap:

Underfitting occurs when the model is too simple to capture the structure in the data. Both training loss and test loss are high, and the generalisation gap is small - the model fails on training data and test data alike. Remedies include increasing model capacity, training longer, or tuning the learning rate and optimiser.

Overfitting occurs when the model is too complex and memorises training data, including its noise. Training loss is low but test loss is high, producing a large generalisation gap. The model has learned patterns that are specific to the training set and do not generalise. Remedies include regularisation (discussed in section 1.3), obtaining more data, or reducing model capacity.

The primary diagnostic tool is monitoring the generalisation gap during training - figure 1.1 shows an example plot of training loss and validation loss together as a function of epochs. When validation loss stops improving and begins to rise while training loss continues to fall, overfitting is occurring.

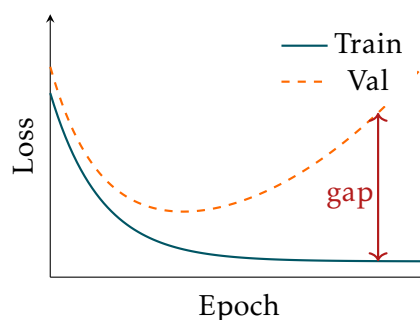


Figure 1.1: Training and validation loss over epochs. Train loss decreases monotonically; validation loss initially decreases then increases as the model begins to overfit. The gap between the two curves is the generalisation gap.

1.2 Generalisation Paradox

Classical statistical learning theory makes a clear prediction: models with more parameters than training examples should overfit catastrophically. Deep networks violate this prediction routinely - they are massively overparameterised yet generalise well. This section examines why, starting from the classical bias-variance framework and then developing the modern understanding of why deep networks are different.

► Session 7:
Bias-Variance
Tradeoff, Limits of
Classical View,
Many Good
Minima, Flat vs
Sharp Minima,
Double Descent

1.2.1 Bias-Variance Tradeoff

Classical statistical learning theory decomposes the expected test error into two competing terms. The **bias** measures how wrong the model is on average across all possible training sets - a high-bias model is too simple to capture the true data-generating process. The **variance** measures how sensitive the model is to the specific training set used - a high-variance model fits each training set very well but produces different predictions for different training sets, so it fails on new data.

The classical prescription is to choose model complexity to balance the two: simple models have high bias and low variance (underfitting), complex models have low bias and high variance (overfitting), and the optimal model sits somewhere in between. Adding more parameters reduces bias but increases variance - so classical theory predicts that overparameterised models should overfit badly.

1.2.2 Limits of the Classical View

Deep networks have huge numbers of parameters - far more than needed to memorise the training data. They routinely drive training loss close to zero, nearly memorising the training set. Classical bias-variance theory predicts this should lead to catastrophic overfitting. Yet in practice, deep networks generalise well. This is the **generalisation paradox** of deep learning, and it is not fully resolved by classical theory. Understanding why deep networks generalise requires looking beyond the classical framework.

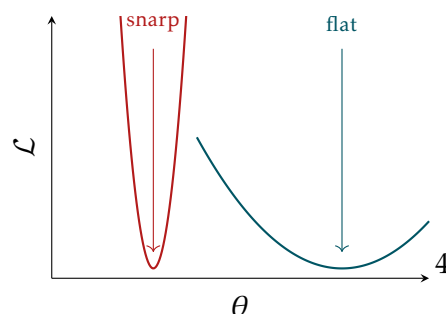
1.2.3 Many Good Minima

One part of the explanation is that overparameterised networks have a vast number of local minima, and most of them have very similar loss values. Networks trained from different random initialisations converge to different minima yet achieve similar generalisation performance. This means the choice of which minimum gradient descent finds is less critical than classical theory suggests - there are simply many equivalent solutions, and the optimiser finds one of them.

A theoretical underpinning for this observation comes from the connection to statistical physics. Choromanska et al. (2015) showed, using a correspondence with spin-glass models, that in deep networks the local minima with high loss become exponentially rare as network size grows - almost all local minima cluster near the global minimum. The more practically relevant obstacle is not local minima at all but **saddle points** - critical points where the loss curves upward in some directions and downward in others. Dauphin et al. (2014) argued that in high-dimensional spaces, saddle points vastly outnumber local minima, and that escaping saddle points is the main challenge for gradient-based optimisation in deep networks. The noise introduced by mini-batch SGD helps here too: stochastic gradient estimates provide perturbations that can push iterates away from saddle regions where the exact gradient would stall.

1.2.4 Flat vs Sharp Minima

Not all minima generalise equally well. A **sharp minimum** is one where the loss changes rapidly as parameters move away from the minimum - the basin is narrow and deep. A **flat minimum** is one



where the loss changes slowly - the basin is wide and shallow. Figure 1.2 illustrates the difference.

The connection between flatness and generalisation was first formalised by Hochreiter and Schmidhuber (1997), who argued using minimum description length that flat minima correspond to simpler, more compressible models and therefore generalise better. The intuition is the following: when we move from the training distribution to the test distribution, the loss surface shifts slightly. A sharp minimum that is optimal for the training distribution may be far from optimal for the test distribution - a small shift in the surface is enough to push the parameters out of the narrow basin, causing a large increase in loss. A flat minimum is more robust: even if the surface shifts, the parameters remain in a region of low loss.

The practical importance of this observation was demonstrated by Keskar et al. (2017), who showed empirically that large-batch SGD consistently converges to sharp minima that generalise poorly, while small-batch SGD finds flat minima that generalise well. This provides a direct explanation for the well-known generalisation gap between large-batch and small-batch training, and confirms that mini-batch SGD acts as an **implicit regulariser** - it biases the optimiser toward solutions that generalise better, without any explicit change to the loss function.

1.2.5 Double Descent

The classical bias-variance curve predicts a single U-shaped relationship between model complexity and test error - one sweet spot of optimal complexity. Belkin et al. (2019) showed that this picture is incomplete: the **double descent** phenomenon reveals that beyond the interpolation threshold, test error decreases again as complexity grows further.

As model complexity grows, test error first follows the classical U-shape. At the **interpolation threshold** - the point where the model has just enough capacity to fit the training data exactly - test error peaks. This is the worst operating point: the model is maximally rigid, forced to commit every degree of freedom to fitting training points exactly, with no slack for smooth interpolation. Beyond this threshold, as complexity continues to grow, test error decreases again. Nakkiran et al. (2020) demonstrated that this phenomenon occurs in deep neural networks specifically, appearing not only as a function of model size but also as a function of training epochs and dataset size.

The intuition for why overparameterisation helps is that excess capacity allows

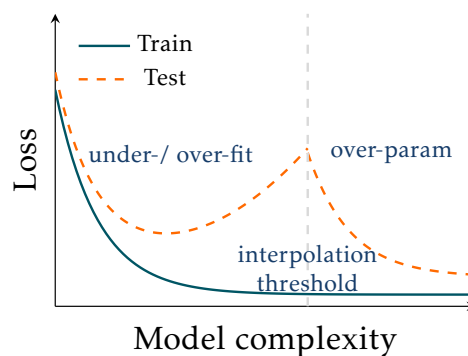


Figure 1.3: Double descent. The classical U-shaped bias-variance curve (left) predicts a single optimal complexity. Beyond the interpolation threshold, test loss decreases again as model complexity grows further.

smoother, more natural interpolation between training points. When a model has just enough parameters to fit the training data, every degree of freedom is fully committed - the interpolant is maximally constrained and can produce sharp, jagged behaviour between training points. When the model is overparameterised, there are infinitely many solutions that fit the data exactly, and gradient descent from a small random initialisation selects among them. Bartlett, Montanari, and Rakhlin (2020) showed for linear models that gradient descent finds the **minimum-norm solution** - the interpolant that fits the training data while keeping the parameter vector as small as possible. Minimum-norm solutions are smooth by construction: large oscillations between training points require large parameter magnitudes, so minimising the norm suppresses them.

This connects back to the **implicit regularisation of mini-batch SGD** discussed in section 1.2.4. The noise in gradient estimates not only favours flat minima over sharp ones, but also biases the optimiser toward low-norm solutions. Both effects work in the same direction: the optimisation procedure itself acts as a regulariser, steering training toward solutions that generalise well - without any explicit penalty term in the loss. Deep networks operate in this overparameterised regime, to the right of the interpolation threshold, and benefit from both effects.

1.3 Regularisation Techniques

SGD provides implicit regularisation for free. We now turn to explicit techniques that can be added on top to further control the generalisation gap.

1.3.1 Weight Decay

Large weights indicate that the model relies heavily on specific features - small changes in input produce large changes in output, increasing sensitivity to the training set and reducing generalisation. The classical fix is to penalise large weights directly by adding a regularisation term to the loss:

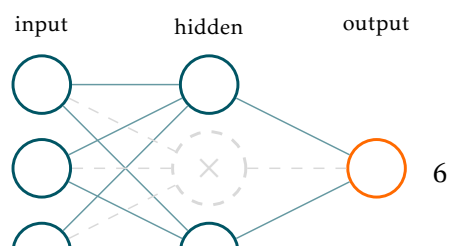
$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \frac{\lambda}{2} \|\theta\|^2. \quad (1.4)$$

This is **L2 regularisation**, also called **weight decay** - the gradient of the penalty term, $\lambda\theta$, shrinks all weights toward zero at each update step, distributing reliance more evenly across features. The hyperparameter $\lambda > 0$ controls the strength and is tuned on the validation set.

In PyTorch, weight decay is available via the `weight_decay` parameter within optimisers. Weight decay is a useful starting point but it is less central in deep learning than in classical ML. Mini-batch SGD already provides implicit regularisation by biasing toward low-norm solutions, reducing the marginal benefit of an explicit penalty. Moreover when using Adam, standard L2 regularisation interacts undesirably with the adaptive learning rates (Loshchilov and Hutter 2019).

1.3.2 Dropout

Dropout (Srivastava et al. 2014) is one of the most powerful and widely used regularisation techniques



► Session 7: Weight Decay, Dropout – Training, Dropout – Evaluation, Dropout – Ensemble Interpretation, Data Augmentation, Early Stopping

in deep learning. It randomly zeros each activation during training with probability p , called the **dropout rate**. Each forward pass effectively uses a different subnetwork - a random subset of the neurons, as illustrated in Figure 1.4.

Dropout prevents neurons from **co-adapting**: no neuron can rely on specific other neurons always being present, because those neurons may be absent in the next forward pass. Each neuron is therefore forced to be independently useful - it must learn a feature that is valuable on its own, not one that only makes sense in combination with specific other neurons. On the other hand, it also prevents **over-specialisation**: neurons are discouraged from learning highly specific, narrow features that are only useful in one context. The result is a network that learns more distributed, robust representations - each neuron contributes meaningfully on its own, and the representations generalise better to unseen data.

Train and evaluation modes. During training, only a fraction $(1 - p)$ of neurons are active on average - the network learns with reduced capacity. During evaluation the behaviour changes in two ways. First, all neurons are active - no masking is applied. Second, the outputs must be rescaled to account for the difference in the number of active neurons between training and evaluation. Without correction, the full network at test time would produce larger activations than the network was trained to expect, changing the effective scale of every layer.

The standard correction is to multiply activations by $(1 - p)$ at test time. An equivalent and more computationally convenient formulation is **inverted dropout**: scale active activations up by $\frac{1}{1-p}$ during training instead, so that no correction is needed at evaluation. This is what PyTorch implements - `nn.Dropout(p)` scales active activations by $\frac{1}{1-p}$ during training and passes activations through unchanged during evaluation. The net effect is the same; the advantage is that the evaluation forward pass requires no extra operations.

Typical values of p range from 0.1 to 0.5, tuned on the validation set. Dropout works best when applied to **fully connected layers**. In other architectures - convolutional networks, recurrent networks, transformers - the standard formulation is less effective or requires adaptation. Dropout also interacts with batch normalisation in a non-trivial way; this is addressed in section 1.4.1.

Ensemble interpretation. A deeper understanding of why dropout works comes from the ensemble perspective. A network with n neurons and dropout rate p implicitly defines 2^n different subnetworks - one for each possible dropout mask. During training, each mini-batch trains a different subnetwork, but all subnetworks share the same weights. This is an extremely parameter-efficient form of ensemble training: instead of training 2^n separate models, a single set of weights is shared across all of them.

At test time, using the full network with rescaled activations approximates averaging the predictions of all 2^n subnetworks simultaneously. Ensemble averaging is a classical and well-understood technique for reducing variance: individual models may overfit in

different directions, but their average tends to cancel out idiosyncratic errors and generalise better. Dropout achieves this ensemble effect at the cost of training and deploying a single model - a remarkable result given the exponential number of subnetworks implicitly represented.

1.3.3 `model.train()` and `model.eval()`

During training we want certain layers to behave stochastically - dropout should randomly zero activations, producing a different subnetwork for each forward pass. During evaluation and inference we want deterministic, stable behaviour - dropout should be disabled and all neurons should be active. These are two distinct operational regimes that require different behaviour from the same layers. Other layers also have dual behaviour - most notably batch normalisation, which we will encounter in section 1.4.1.

PyTorch implements this through a **model mode**: every `nn.Module` instance is either in *training mode* or *evaluation mode*. The mode is set by calling `model.train()` or `model.eval()` on your instantiated model, where `model` is whatever name you gave it. Both calls propagate recursively to all submodules, so a single call on the top-level model is sufficient regardless of how many layers it contains. Layers that respond to the mode switch include `nn.Dropout` and, as we will see later, `nn.BatchNorm`; all other standard layers - `nn.Linear`, `nn.ReLU`, and so on - are unaffected.

The correct pattern in a training loop is:

```
model = MyNetwork(...)          # your instantiated model

# training loop
model.train()
for X_batch, y_batch in train_loader:
    optimizer.zero_grad()
    loss = loss_fn(model(X_batch), y_batch)
    loss.backward()
    optimizer.step()

# validation loop
model.eval()
with torch.no_grad():
    for X_batch, y_batch in val_loader:
        loss = loss_fn(model(X_batch), y_batch)
```

Remark 2. Forgetting to call `model.eval()` before validation or inference is a silent bug - no error is raised, but predictions are wrong because dropout is still active and batch normalisation uses incorrect statistics. Always call `model.eval()` before validation and `model.train()` before resuming training.

1.3.4 Data Augmentation

Data augmentation **creates new training examples** by applying transformations to existing ones from the training set. The fundamental requirement is **invariance**: the

correct prediction must be invariant to the transformation, which means the transformation must be **label-preserving**. These are two ways of saying the same thing - if rotating an image of a cat still produces an image of a cat, the model should be invariant to rotation, and rotation is therefore a valid label-preserving transformation.

This constraint gives augmentation two complementary benefits. First, it **increases the effective size of the training set** - more diverse training examples reduce overfitting regardless of their content. Second, and more fundamentally, it **explicitly teaches the model the invariances that matter for the problem**: a model trained on cats in many orientations learns that orientation does not affect the label; a model trained on patient measurements with small added noise learns that minor measurement variation does not affect the diagnosis.

For image data, common augmentations include random horizontal flips, rotations, crops, and colour jitter; for tabular data, Gaussian noise on continuous features and random feature masking are typical choices. Though these are reasonable starting points, they are not universally valid.

The choice of transformations requires **domain knowledge** - only the practitioner knows which invariances are valid for their specific problem. Every augmentation is an implicit claim about the data-generating process: it asserts that transformed examples are plausible draws from p_{data} with the same label. A horizontal flip is valid for natural image classification - a cat seen from the left is the same cat seen from the right. The same flip applied to digit recognition is wrong - a 6 rotated 180° becomes a 9. Aggressive colour changes are harmless for classifying animals but damaging for medical imaging, where colour carries diagnostic information. **Augmentation pipelines from papers and tutorials must never be transferred blindly to a new problem.** Every augmentation must be justified by asking: is this transformation label-preserving for *my* problem and *my* data?

In practice, augmentation is applied **on the fly during data loading**: at each training epoch, transformations are sampled randomly and applied to training examples as they are loaded into a mini-batch. The training set itself is never modified - the model sees a different random version of each example at every epoch, continuously expanding the effective variety of training data.

Note that invariance can also be built directly into the architecture rather than learned from augmented data - certain architectures are invariant to specific transformations by construction. This will become relevant when we study convolutional networks.

1.3.5 Early Stopping

Training longer always reduces training loss - the optimiser will keep finding ways to fit the training data more precisely. But beyond a certain point this comes at the cost of generalisation: the model begins to memorise training-specific patterns that do not hold for the population, and the generalisation gap grows. The validation loss, which tracks performance on held-out data, reveals this: it initially decreases alongside training loss, but eventually levels off and begins to rise as overfitting sets in, as illustrated in Figure 1.5.

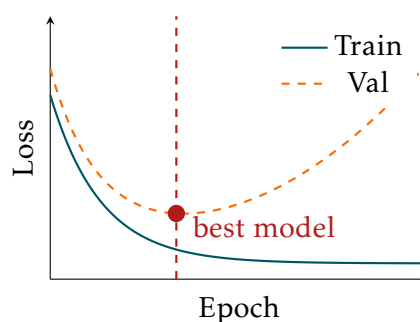


Figure 1.5: Early stopping. Training loss decreases monotonically; validation loss eventually increases. The best model is saved at the validation minimum and deployed rather than the final model.

The natural response is to stop training at the point where the model generalises best - the minimum of the validation loss curve - rather than training until convergence. This is **early stopping**. It is a form of regularisation because it constrains the effective training time: a model trained for fewer steps has had less opportunity to memorise the training data, limiting its effective capacity without changing the model architecture or the loss function.

In practice, the best model encountered during training is saved at each epoch and the final deployed model is this saved checkpoint, not the model at the end of training. Since validation loss is noisy and may fluctuate temporarily before resuming its descent, training is not stopped at the first sign of non-improvement. Instead, a **patience** parameter k specifies how many consecutive epochs without improvement to tolerate before stopping. All the ingredients for early stopping are already present in the training loop from session 6 - it simply adds a stopping criterion and model checkpointing.

1.4 Training Stability

Regularisation solves the problem of overfitting - it allows us to use large, overparameterised networks without the model memorising the training data. But deeper networks introduce a separate class of problems that have nothing to do with overfitting: **training instabilities**. These are problems that make training impossible or unreliable regardless of whether the model would overfit. More layers amplify small issues, and three instabilities are particularly important: internal covariate shift, vanishing and exploding gradients, and the dying ReLU problem.

► Session 7:
Training Stability,
Internal Covariate
Shift, Batch
Normalisation,
Vanishing and
Exploding
Gradients, Dying
ReLU, Activation
Functions

1.4.1 Internal Covariate Shift and Batch Normalisation

In section ?? we saw that normalising the inputs to the network stabilises training: unnormalised features with very different scales produce an ill-conditioned loss surface and make gradient descent behave poorly. The same problem occurs at every layer of a deep network, not just the first one - and it is aggravated by the fact that training is a dynamic process.

Consider a layer l receiving activations from layer $l - 1$ as inputs. The distribution of outputs of layer l depends on two things: the data and the parameters of all preceding layers. After a gradient update, both change simultaneously. The parameters of layer l and all preceding layers update, changing the distribution of outputs each layer produces affecting all subsequent layers down the road. This effect compounds with depth: a layer deep in the network receives inputs that have been transformed through many layers, each of which updated its parameters, so the distribution it sees can shift dramatically between update steps. The non-linearity amplifies this further - a shift in the pre-activation distribution can produce a very different post-activation distribution, potentially pushing activations into saturating regions where gradients vanish, or dramatically increasing the variance of the activations. This phenomenon of changing distributions within layers is called **internal covariate shift** (Ioffe and Szegedy 2015).

The consequences are the same as for unnormalised inputs: slower convergence, need for lower learning rates, and high sensitivity to initialisation. The solution follows the same logic too - if normalising the inputs to the network stabilises the first layer, normalising the activations at every layer stabilises all of them. This is exactly what **batch normalisation** does: it explicitly normalises the activations at each layer to have zero mean and unit variance within each mini-batch, then applies learnable affine transformations:

$$\hat{h}_j^{(i)} = \frac{h_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}, \quad \tilde{h}_j^{(i)} = \gamma_j \hat{h}_j^{(i)} + \beta_j, \quad \mu_j = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} h_j^{(i)}, \quad \sigma_j^2 = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (h_j^{(i)} - \mu_j)^2 \quad (1.5)$$

where μ_j and σ_j^2 are the mean and variance of feature j computed across all examples $i \in \mathcal{B}$ in the current mini-batch, ϵ is a small constant for numerical stability, and γ_j and β_j are learnable per-feature scale and shift parameters that allow the normalised activation to be rescaled and shifted. By keeping the input distribution to each layer approximately stable, batch normalisation allows higher learning rates, reduces sensitivity to initialisation, and makes training of very deep networks practical.

During training, the normalisation uses statistics computed from the current mini-batch - mean and variance are different for each batch, injecting a small amount of noise into the normalisation. This is analogous to the noise in mini-batch SGD and has a **mild implicit regularisation effect**, though this is not the primary motivation for using batch normalisation.

During evaluation, mini-batch statistics are replaced by **running statistics** - exponential moving averages of $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ accumulated during training. This gives deterministic, stable behaviour at test time: the normalisation no longer depends on which other examples happen to be in the same batch. This is why batch normalisation is one of the layers affected by `model.train()` and `model.eval()` - forgetting to call `model.eval()` before inference means the network normalises using noisy mini-batch statistics rather than the stable running statistics, producing wrong and irreproducible predictions.

In PyTorch, `nn.BatchNorm1d` is used for fully connected networks and `nn.BatchNorm2d` for convolutional networks - covered in a later session. Batch norm is typically inserted after the linear transformation and before the activation function.

Batchnorm and Dropout. Using batch norm and dropout together requires care. The fundamental problem is a **variance shift** (Li et al. 2019): during training with dropout active, some activations are zeroed, reducing the variance of the distribution entering batch norm. The running statistics accumulated during training therefore reflect this reduced variance. At test time, dropout is disabled and all activations are present - the variance is higher, but the running statistics used for normalisation were computed under the lower-variance training distribution. This mismatch produces systematically wrong normalisation at test time and degrades performance. In practice, three approaches are used:

- **Batch norm only** - sufficient in many cases, particularly in convolutional networks where batch norm provides strong regularisation on its own
- **Dropout only** - effective in MLPs with no interaction problems
- **Both** - possible if dropout is placed *after* batch norm, so that running statistics are computed on the full activations; results may still be suboptimal and require

careful tuning

An alternative that avoids these complications entirely is **layer normalisation** (Ba, Kiros, and Hinton 2016). Rather than normalising across the batch dimension per feature, layer norm normalises across the feature dimension per example:

$$\hat{h}_j^{(i)} = \frac{h_j^{(i)} - \mu^{(i)}}{\sqrt{\sigma^{2(i)} + \epsilon}}, \quad \tilde{h}_j^{(i)} = \gamma_j \hat{h}_j^{(i)} + \beta_j \quad \mu^{(i)} = \frac{1}{D} \sum_{j=1}^D h_j^{(i)}, \quad \sigma^{2(i)} = \frac{1}{D} \sum_{j=1}^D (h_j^{(i)} - \mu^{(i)})^2 \quad (1.6)$$

where $\mu^{(i)}$ and $\sigma^{2(i)}$ are the mean and variance computed across all D features of example i , ϵ is a small constant for numerical stability, and γ_j and β_j are learnable per-feature scale and shift parameters. Because statistics are computed per example independently, there are no running statistics to accumulate and no train/eval difference - layer norm behaves identically in both modes. This means layer norm and dropout combine cleanly. In MLPs, the typical order is: `nn.Linear` $\rightarrow$ `nn.LayerNorm` $\rightarrow$ activation $\rightarrow$ `nn.Dropout`. Transformers use a related but slightly different pattern involving residual connections - this will be covered when we study sequence models.

Batch norm and layer norm are two points in a larger design space of normalisation techniques. **Group norm** normalises across groups of channels per example and is useful when batch sizes are small. **RMS norm** is a simplified layer norm without mean subtraction, used in modern large language models such as LLaMA. The field has not settled on a single winner - the right choice depends on the architecture, batch size, and task.

1.4.2 Vanishing and Exploding Gradients

The second major training instability arises during backpropagation. The gradient of the loss with respect to the parameters of the first layer involves a product of Jacobians across all layers:

$$\frac{\partial \mathcal{L}}{\partial \theta^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \cdot \frac{\partial \mathbf{h}^{(L)}}{\partial \mathbf{h}^{(L-1)}} \cdots \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \theta^{(1)}}. \quad (1.7)$$

This product of L Jacobian matrices compounds as it travels backwards from the loss to the earliest layers. If the singular values of the Jacobians are consistently less than 1, the product shrinks exponentially with depth - **vanishing gradients**. The earliest layers receive gradients close to zero, their parameters barely update, and the deep network effectively behaves as a shallow one. If the singular values are consistently greater than 1, the product grows exponentially - **exploding gradients**. The earliest layers receive enormous gradients, a single update step moves the parameters by a huge amount, and training diverges.

The solutions discussed in Session 5 address this directly. **Xavier and He initialisation** choose the variance of the initial weights to keep the Jacobian singular values close to 1 at the start of training. **Batch normalisation** stabilises the forward pass distributions, which indirectly stabilises the backward pass. **Gradient clipping** is a runtime solution for exploding gradients: if the gradient norm exceeds a threshold τ , the gradient is rescaled to have norm exactly τ before the update step. In PyTorch: `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`, called between `loss.backward()` and `optimizer.step()`. Gradient clipping is standard practice in recurrent network and Transformer training, where exploding gradients are particularly common.

1.4.3 Dying ReLU and Activation Functions

The third instability is the **dying ReLU** problem. The ReLU activation function has a zero gradient for all negative pre-activations:

$$\frac{\partial \text{ReLU}(z)}{\partial z} = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0. \end{cases} \quad (1.8)$$

This sounds alarming: ReLU has zero gradient for half the input space. In a deep network, the vanishing gradient problem would seem to be inevitable.

In practice, the dying ReLU problem is less severe than it first appears. A neuron dies only if its pre-activation is negative for *all* training examples simultaneously. In a diverse dataset, pre-activations vary across examples - some positive, some negative - and the average gradient over a mini-batch is nonzero. The diversity of the training data saves us. Moreover, He initialisation is specifically designed to keep pre-activations in a healthy range initially, further reducing the risk. ReLU remains the recommended baseline activation function: it is simple, computationally cheap, and works well in practice.

When dying ReLU does become a problem, the fix is to use an activation function that has a nonzero gradient for negative inputs. **Leaky ReLU** introduces a small negative slope α :

$$\text{LeakyReLU}(z) = \max(\alpha z, z), \quad \alpha \ll 1. \quad (1.9)$$

ELU (Exponential Linear Unit) uses a smooth exponential curve for negative inputs, avoiding the hard zero entirely. More recent activations - **GELU**, **Swish**, and **Mish** - are smooth, differentiable everywhere, and are used in modern architectures such as Transformers. There is no universal winner; ReLU and Leaky ReLU cover most practical needs for the feedforward networks considered in this course.

1.5 Embracing Randomness

A recurring pattern throughout this lecture is that randomness is not merely a technical nuisance to be eliminated but a fundamental feature of deep learning training that contributes to generalisation. Weight initialisation is random - it determines which region of the loss landscape is explored. Data shuffling at the start of each epoch introduces randomness into the batch composition, preventing the model from memorising the order of examples. Mini-batch SGD introduces noise into the gradient estimates, which implicitly regularises by favouring flat minima. Dropout intentionally injects randomness into the forward pass, producing an ensemble effect. Data augmentation applies random transformations to training examples, expanding coverage of the population distribution.

This perspective inverts the usual framing. Rather than asking how to reduce randomness and achieve more deterministic training, the question becomes: how can we design the right kinds of randomness to improve generalisation? The techniques in this lecture - from the implicit noise of mini-batch SGD to the explicit masking of dropout to the domain-informed transformations of augmentation - are all answers to this question.

Acknowledgements

These lecture notes were prepared with the assistance of Claude, a large language model developed by Anthropic. All content has been reviewed, edited, and approved by the author.

References

- Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton (2016). “Layer Normalization”. In: *arXiv preprint arXiv:1607.06450*.
- Bartlett, Peter L., Andrea Montanari, and Alexander Rakhlin (2020). “Benign Overfitting in Linear Regression”. In: *Proceedings of the National Academy of Sciences*. Vol. 117. 48, pp. 30063–30070.
- Belkin, Mikhail et al. (2019). “Reconciling Modern Machine-Learning Practice and the Classical Bias-Variance Trade-off”. In: *Proceedings of the National Academy of Sciences* 116.32, pp. 15849–15854.
- Choromanska, Anna et al. (2015). “The Loss Surfaces of Multilayer Networks”. In: *Proceedings of the 18th International Conference on Artificial Intelligence and Statistics*.
- Dauphin, Yann et al. (2014). “Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization”. In: *Advances in Neural Information Processing Systems*.
- Hochreiter, Sepp and Jürgen Schmidhuber (1997). “Flat Minima”. In: *Neural Computation* 9.1, pp. 1–42.
- Ioffe, Sergey and Christian Szegedy (2015). “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. In: *Proceedings of the 32nd International Conference on Machine Learning*, pp. 448–456.
- Keskar, Nitish Shirish et al. (2017). “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima”. In: *International Conference on Learning Representations*.
- Li, Xiang et al. (2019). “Understanding the Disharmony between Dropout and Batch Normalization by Variance Shift”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2682–2690.
- Loshchilov, Ilya and Frank Hutter (2019). “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations*.
- Nakkiran, Preetum et al. (2020). “Deep Double Descent: Where Bigger Models and More Data Hurt”. In: *International Conference on Learning Representations*.
- Srivastava, Nitish et al. (2014). “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.1, pp. 1929–1958.